

Probeable Problems for Beginner-level Programming-with-AI Contests

Mrigank Pawagi
mrigankp@iisc.ac.in
Indian Institute of Science
Bengaluru, Karnataka, India

Viraj Kumar
viraj@iisc.ac.in
Indian Institute of Science
Bengaluru, Karnataka, India

ABSTRACT

To broaden participation, competitive programming contests may include beginner-level problems that do not require knowledge of advanced Computer Science concepts (e.g., algorithms and data structures). However, since most participants have easy access to AI code-generation tools, these problems often become trivial to solve. For beginner-friendly programming contests that do not prohibit the use of AI tools, we propose *Probeable Problems*: code writing tasks that provide (1) a problem specification that deliberately omits certain details, and (2) a mechanism to probe for these details by asking clarifying questions and receiving immediate feedback. To evaluate our proposal, we conducted a 2-hour programming contest for undergraduate Computer Science students from multiple institutions, where each student was an active member of their institution’s ACM student chapter. The contest comprised of six Probeable Problems for which a popular code-generation tools (e.g., GitHub Copilot) were unable to generate accurate solutions due to the absence of details. Students were permitted to work individually or in groups, and were free to use AI tools. We obtained consent from 26 groups (67 students) to use their submissions for research. To determine whether Probeable Problems are suitable for such contests, we analyze the extent to which the code submitted by these groups identifies missing details.

CCS CONCEPTS

• **Social and professional topics** → **Informal education**; • **Software and its engineering** → *Requirements analysis*.

KEYWORDS

Ambiguity, Code specifications, Code writing, CS1

ACM Reference Format:

Mrigank Pawagi and Viraj Kumar. 2024. Probeable Problems for Beginner-level Programming-with-AI Contests. In *ACM Conference on International Computing Education Research V.1 (ICER '24 Vol. 1)*, August 13–15, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3632620.3671108>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICER '24 Vol. 1, August 13–15, 2024, Melbourne, VIC, Australia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0475-8/24/08
<https://doi.org/10.1145/3632620.3671108>

1 INTRODUCTION

Participants in competitive programming contests typically solve a series of code-writing tasks within a stipulated time. The premier competitive programming contest is the International Collegiate Programming Contest (ICPC), whose purpose¹ is “to advance prospects for the next generation by bringing students together, working collaboratively to solve *algorithmically challenging problems*, and preparing them to build dependable systems that benefit their neighbors with the support of universities, industry, and community leaders, globally” (emphasis added). To welcome beginners to the exciting world of competitive programming, many contests, including those organized by popular websites such as LeetCode, AtCoder, and CodeForces, have traditionally offered beginner-level code-writing problems that require no knowledge of advanced Computer Science concepts in algorithms and data structures. Today, most participants have easy access to AI code-generation tools that can often solve beginner-level code-writing problems [11, 19, 20, 40, 45]. While the use of such tools may be banned by some contest organizers, others may permit them because they are unwilling or unable to prohibit their use. Thus, our study is motivated by the following question:

Is it possible to design beginner-level competitive programming problems that *cannot* be solved successfully by AI code-generation tools alone?

For instance, since the large language models (LLMs) underlying modern code-generation tools are expensive to retrain and are known to be sensitive to subtle changes in the format of their prompts [42], it may be feasible to create novel (or seemingly-novel) variants of beginner-level problems to foil such tools. In a recent study, Huang et al. [23] showed that GPT-4 excels at “easy” code-generation problems from CodeForces contests conducted prior to September 2021 (the cut-off for GPT-4’s training²), but this performance falls sharply thereafter. The authors define “easy” problems as those between 800 to 1,100 on the CodeForces difficulty ratings for problems. However, Jain et al. [25] argue that beginner-level (Divisions 3 and 4) CodeForces problems are more challenging than “easy” problems from LeetCode and AtCoder. Hence, they define “easy” CodeForces problems as those with the lowest possible rating (800), and “hard” problems as those rated over 1,000. Based on this scale, Jain et al. maintain a live leaderboard showing code-generation performance of several LLMs on newly released contest problems³. At the time of writing, the best-performing models successfully solves over 85% of “easy” problems from the most recent contests.

¹<https://icpc.global/worldfinals/fact-sheet/ICPC-Fact-Sheet.pdf>

²<https://platform.openai.com/docs/models/gpt-4-and-gpt-4-turbo>

³<https://livecodebench.github.io/leaderboard.html>

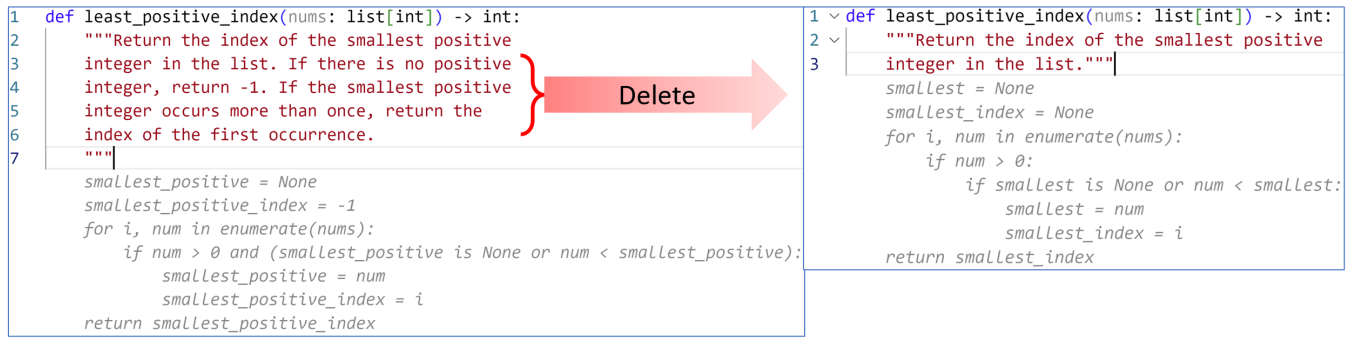


Figure 1: (Left, full specification) A screenshot showing a correct solution suggested by GitHub Copilot (grey font, after Line 7). **(Right, after trimming away details)** GitHub Copilot suggests a functionally *inequivalent* solution (after Line 3), since it is forced to make “reasonable” decisions about the omitted details.

To avoid ambiguity, the task specifications for competitive programming problems are fully detailed. In contrast, real-world software specifications can be ambiguous in a variety of ways [5, 15, 17, 27, 29]. To successfully thwart AI code-generation tools on beginner-level code-writing tasks, we draw inspiration from two sources. The first source is a specific type of ambiguity that Schneider called for in 1978 [41]. He argued that CS1 students “should be able to recognize and resolve uncertainties in simple problem statements”. Further, as one way to develop this ability, he suggested that “some programming assignments should *intentionally be left incomplete*, requiring the student to *consider the alternatives* and to *make a reasonable decision* on the omitted details” (emphasis added). Our second source of inspiration is the eagerness that current AI tools display in generating code even when the specification is incomplete. Figure 1 shows that an AI code-generation tool (GitHub Copilot) can easily generate a correct solution when presented with the full specification (left). This is consistent with prior research on the effectiveness of tools like GitHub Copilot when they are provided full specifications for CS1 code-writing problems [11, 47]. However, when key details from this specification are omitted, the tool nevertheless attempts to generate code. In doing so, it makes *silent* decisions (which may or may not be reasonable in Schneider’s sense) to resolve the ambiguities created by omitting details.

1.1 Probeable Problems

A Probeable Problem has two components: (1) a code-writing task to create a function given an *incomplete* specification, and (2) an oracle that reveals the function’s desired behavior on any input(s) chosen by the contestant. The contestant must recognize on their own that the specification allows more than one *possible* result on a certain input before asking the oracle for the desired result on that input. While the oracle denies contestants an opportunity to make their own decisions regarding the details omitted by the specification, we believe that this is necessary for two reasons: it imposes no additional burden on contest organizers, and it keeps the problem difficulty level the same for all contestants.

Figure 2 shows an example of a Probeable Problem, whose task specification deliberately omits two details. First, it does not specify what the function should return when the input list contains no

positive integers (e.g., when it is empty). Second, it does not specify which index the function should return if the smallest positive integer occurs more than once in the list. When presented with such a specification, what might a contestant do? We consider three among several possibilities in the following vignettes.

Fictitious Contestant 1. This contestant fails to recognize that the specification lacks detail. Next, as shown in Figure 1 (right), they rewrite the given information as a function signature and docstring and they generate a solution using an AI code-generation tool. Finally, they submit this solution after satisfying themselves that it corresponds to their (incorrect) understanding of the task.

Fictitious Contestant 2. This contestant mimics Contestant 1 up to the point where they have generated the code in Figure 1 (right). Then, while examining the generated code, they realize that it returns `None` when the input is an empty list. Further, they review the specification and confirm that it fails to specify the return value on this input. Next, they ask the oracle for the desired return value on input `[]`, as shown in Figure 2. They replace the AI-generated statement `smallest_index = None` with `smallest_index = -1` and submit the code, believing that they have solved the problem correctly. (They have not.)

Fictitious Contestant 3. This contestant immediately recognizes that the function’s desired return value is unspecified for the common “corner case” input `[]`. They query the oracle as above and then mimic Contestant 1 by writing the function’s signature and docstring. However, based on the oracle’s output, they add an additional sentence to the docstring: If the list is empty, return -1. Next, they use an AI tool to generate code that is identical to the code in Figure 1 (right) except that `smallest_index` is initialized to -1 instead of `None`. While reviewing the code, they realize that the specification was incomplete not only for the specific list `[]`, but for the whole category of lists that contain no positive integers. Thus, they query the oracle again, this time on the input `[0]`. The oracle reports the desired return value as -2. Somewhat mystified, the contestant queries the oracle again with inputs `[-1]` (return value: -2) and `[0, -1]` (return value: -3). At this point, the contestant inductively guesses a pattern: the return value is $-(\text{len}(\text{nums}) + 1)$. They use this expression in place of -1 to

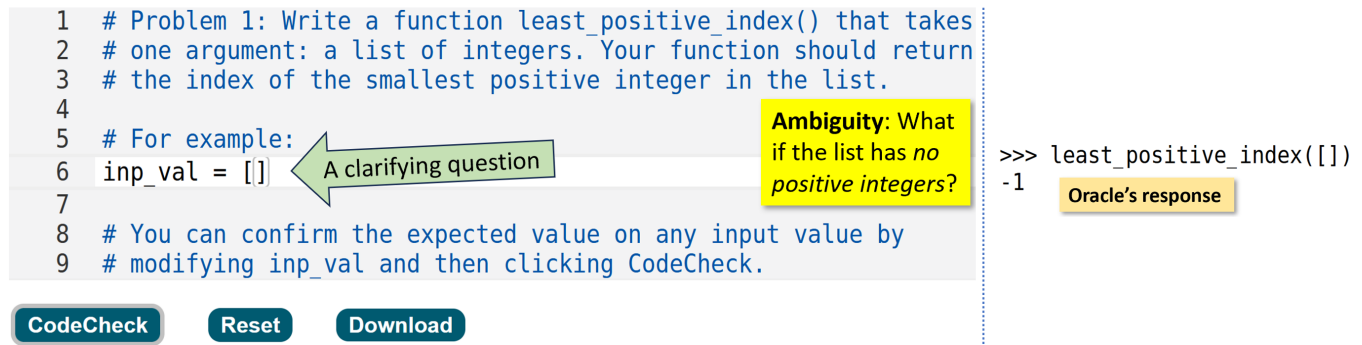


Figure 2: (Left) A Probeable Problem with omitted specification details, and a contestant’s clarifying question attempting to resolve one such detail. (Right) The automated feedback generated by an ‘oracle’ specifies the desired output on this input.

initialize `smallest_index`. They now submit the code, failing to recognize that the specification was incomplete in another respect that causes their code to fail on inputs such as `[1, 1]`.

1.2 Designing Probeable Problems

For our study, we have attempted to design a set of Probeable Problems that omit details of diverse types. We used one of these as a Practice Problem to introduce contestants to this style of problem, as described in Section 3. The remaining five Probeable Problems were used for the contest. Prior to the contest, we ensured that none of these popular AI code-generation tools was able to generate correct solutions for these problems, when prompted as shown in Figure 1 (right): GitHub Copilot, ChatGPT-3.5, and Codeium.

In our initial experiments with designing Probeable Problems, we found that some students started querying the oracle for the expected behaviour on unusual inputs (e.g., strings or lists containing floating-point numbers for a function whose argument was a list of integers). To limit the scope of such exploration, we have designed our oracles to give an appropriate error message on such inputs.

Since we are unaware of prior work that categorizes various types of omitted code-writing specification details, we draw from our own experience in failing to design fully-specified code-writing tasks (e.g., for traditional code-writing assignments). Our six Probeable Problems illustrate four types of omissions. For each problem, we began our design with a specification for a code-writing task that was fully detailed (to the best of our knowledge), and then judiciously omitted⁴ some details of the following types:

1.2.1 Definitions. In each of four problems, we entirely omit the detailed definition of one key term from the specification. For the Practice Problem, this term is “palindrome”. For clarity, this term is highlighted in grey in Figure 3 and the omitted definition is shown in a grey box. We expect CS students (and LLMs) to recall a familiar definition of this term: a string that is precisely equal to its reverse (e.g., ‘racecar’). However, the omitted details expand this definition to include strings such as ‘race car’ and ‘Racecar’. In contrast, we do not expect CS students or LLMs to associate the grey-highlighted terms in Problem 2 (Figure 5) and Problem 4 (Figure 7) with any specific definition. In Problem 5 (Figure 8), the

grey-highlighted term “number” frequently refers to either integer or floating-point values, and we omit the fact that it refers to both types here.

1.2.2 Behavior on certain inputs. Ideally, the specification should describe the function’s expected behavior on each possible input. However, we sometimes fail to describe the expected behavior on certain inputs. For Problem 4, one such omission corresponds to a specific input (the empty list). We show such an omission within a red box with dashed lines (Figure 7). A student who recognizes such an omission can easily ask the oracle for the expected behavior on that specific “corner case” input.

In contrast, if the missing details correspond to a whole class of inputs, these omissions are usually shown within a red box with solid lines. (An exception is described in the next paragraph.) For example, the given Problem 1 specification (Figure 4) omits details not only for the empty list but for any input list that has no positive integers (e.g., `[0, -1]`). Since the oracle only reveals the expected behavior on *specific* inputs, in such cases contestants must query the oracle often enough to *inductively guess* the expected behavior on all inputs for which details are omitted.

In our experience, we have often observed a particular sub-type of this type of omission: when the given specification explicitly describes the expected behavior for a certain category of inputs but neglects to do so for a closely related category. This reminds us of the artistic term *chiaroscuro*⁵, and we use an alternate form of highlighting for such omissions. As an example, the given specification for Problem 4 (Figure 7) draws attention to *positive* integers in the list (highlighted in plain yellow), but fails to clarify the expected behavior when the list contains *non-positive* values (highlighted in *chiaroscuro* yellow).

1.2.3 Return type(s). In two problems, we omit details that would clarify the function’s exact return type(s). In Problem 2, the desired return type could conceivably have been `str` instead of the omitted type `int` (shown within a blue box in Figure 5). Similarly, the two values `buy` and `sell` could have been returned as a list instead of a tuple, as shown in Figure 7. This function has two return types: on the empty list, it returns `None` (of type `NoneType`).

⁴See Section 2.3 for prior work that influences our selection of which details to omit.

⁵<https://en.wikipedia.org/wiki/Chiaroscuro>

1.2.4 Tie-break mechanism(s). In each of three problems, the definite article *the* is used by the given specification to *uniquely* describe the expected behavior for inputs with specific properties. However, for inputs where these properties do not hold, more than one behavior is possible and the specification neglects to clarify how to break ties in such cases. For example, in Problem 3 (Figure 6), the cyan-highlighted expression “the integer” is unambiguous only for lists where a unique integer appears least often. The specification omits details (shown in the cyan box) that describe a tie-break mechanism when more than one integer appears least often in the list. While the tie-break mechanism can sometimes be described in just one word (e.g., for Problem 1 in Figure 4), it may be extremely complex (e.g., for Problem 4 in Figure 7). In such cases, it may be challenging for contestants to probe the oracle with inputs that correspond to various tie-break scenarios.

1.3 Research Questions

Since our questions are designed for a competitive programming contest, they are meant to be challenging. However, as we describe in Section 4, submissions were evaluated based on which omitted details contestants were successfully able to account for in their code. We expect certain types of omissions (e.g., return types or the expected behavior on *specific* inputs) to be easier to probe for than others (e.g., definitions). Even for the same type of omission, the difficulty of understanding the expected behavior via probing is likely to depend on the nature of the problem. For example., the tie-break condition for Problem 1 (Figure 4) is likely to be far easier to deduce than the tie-break condition for Problem 4 (Figure 7). Hence, we analyze contestant performance at the level of each (problem, omission) pair.

For each Probeable Problem, contestants are obliged to probe the oracle on a series of carefully crafted inputs before they can inductively guess the expected behavior, as illustrated in the vignette for Fictitious Contestant 3 (Section 1.1). As illustrated in that vignette, contestants may succeed in inductively guessing the missing details for one omission, but fail to recognize further omitted details. To provide some assistance, contestants we given access to verifiers for each Probeable Problem after submitting their first attempts. (We defer further details on these verifiers to Section 4). Contestants could use these verifiers as often as they wished to check the correctness of subsequent versions of their code, before submitting their second (and final) attempts.

Our research questions are as follows:

- RQ1** How successful were contestants at determining the missing details for each (problem, omission) pair?
- RQ2** Were contestants able to leverage the verifiers to identify additional omissions?

2 RELATED WORK

We position our work in relation to three major strands of prior research. First, since many educators are concerned that beginners will develop an over-reliance on AI code-generators [4] and may lose their coding skills [13] (skills that competitive programming has traditionally sought to bolster), we consider alternatives to traditional textual specifications that have been proposed for

code-writing tasks (Section 2.1). Next, we contrast competitive programming contests with Probeable Problems against other types of competitive events that also expect contestants to consider the behavior of code on a variety of inputs (Section 2.2). Finally, in Section 2.3 we compare our approach of allowing students to test their understanding of the specification using oracles with other ways of testing such understanding that have been proposed. We also review prior work on how students develop tests, and how educators can help students develop more effective tests.

2.1 Alternative Specifications

As noted previously, a number of AI code-generation tools are easily available for students (e.g., ChatGPT, GitHub Copilot). These tools can often translate natural-language specifications of beginner-level code-writing tasks into accurate solutions [11, 19, 20, 40, 45]. These tools may generate accurate solutions for more complex tasks as well, particularly if students decompose these into simpler sub-tasks [11]. Significantly more advanced code-generation tools are being developed. Since these are less readily available to students at present, we defer a discussion of these tools to Section 6.2.

To foil the usage of such AI-generation tools, researchers have recently experimented with replacing textual specifications with visual specifications. Rojit et al. originally proposed such specifications for web-application programming to aid specification comprehension in contexts where “learners or instructors lack fluency in the language of communication” [39]. More recently, visual specifications have been used to design Prompt Problems [12], where an illustration shows how specific input values should be converted into outputs. To solve such a problem, students submit a general prompt that is inductively deduced from these examples [34]. The quality of this submission is assessed by using a code-generator to translate the generalized prompt into code, which is then compared against the instructor’s reference solution.

There are two reasons why specifications that represent input-output examples visually are unlikely to foil AI code-generation tools in the context of beginner-level competitive programming. First, Ventakesh et al. [45] show that many such problems can be solved by providing meaningful names of functions and arguments, and a small number of input-output examples. The latter can be easily transcribed from visual specifications. Second, as visual transformers and other multi-modal foundation models continue to improve [50], students now have access to AI-tools that can directly convert visual specifications into code⁶.

Further, unlike visual specifications where the examples are pre-defined, the oracle for a Probeable Problem allows students to clarify the expected behavior on any input. In this respect, Probeable Problems are similar to Executable Examples [48]. This connection is explored further in Section 2.3.

2.2 Alternatives to Competitive Programming

While competitive programming typically focuses on writing code given detailed task specifications, prior work explored an alternative contest form called SCORE that touches “all facets of software

⁶OpenAI has made one such multi-modal model (GPT4-o) freely accessible: <https://openai.com/index/hello-gpt-4o/>

Write a function `is_palindrome()` that takes one argument: a string. It checks whether the string is a **palindrome**.

A palindrome is defined as a string which is the same as its reverse after ignoring space characters and ignoring the difference between upper-case and lower-case letters.

Figure 3: Practice Problem. The given task specification deliberately omits the definition of “palindrome” (highlighted in grey). The omitted definition is non-standard (shown in the grey box). CodeCheck: [\[oracle\]](#), [\[verifier\]](#).

Write a function `least_positive_index()` that takes one argument: a list of integers. Your function should return **the largest index** of the smallest positive integer in the list `x`.

If no such index exists, return `-(len(x) + 1)`.

Figure 4: Problem 1. The task specification refers to “the index” (highlighted in cyan) without clarifying how to break ties, if any. The tie-break mechanism “largest” (shown in the cyan box) is deliberately omitted. The value to be returned when the list has no positive integers (shown in the red box) is also deliberately omitted. CodeCheck: [\[oracle\]](#), [\[verifier\]](#)

Write a function `first_positive_integer()` that takes one argument: a string. Your function should return **an int:** the first positive integer in the string.

The first positive integer in the string is defined as the longest integer prefix of the first word in the string that starts with a non-zero digit. Words are separated by space characters.

If no such word exists, return `0`.

Figure 5: Problem 2. The task specification deliberately omits the return type (shown in the blue box) and a definition for the grey-highlighted term “first positive integer”. This definition is non-trivial, and is shown in the grey box. The value to be returned when the string has no such positive integer (shown in the red box) is also deliberately omitted. CodeCheck: [\[oracle\]](#), [\[verifier\]](#)

Write a function `min_freq()` that takes one argument: a non-empty list of integers. Your function should return **the integer** that appears least often in the list.

If there is more than one such integer, return the smallest such integer.

Figure 6: Problem 3. The task specification refers to “the integer” (highlighted in cyan) without clarifying how to break ties, if any. The tie-break mechanism (shown in the cyan box) is deliberately omitted. CodeCheck: [\[oracle\]](#), [\[verifier\]](#)

development, from requirements analysis and elicitation to implementation and verification” [30]. Similar to Probeable Problems, tasks in a SCORE contest are poorly specified. To obtain clarity, contestants must frequently interact with a *human* “stakeholder” (vs. an automated “client” oracle in our case), and this stakeholder plays a key role in ensuring that contestants devote “considerable attention to user needs and requirements elicitation” [30]. Further, a SCORE contest is organized over several weeks and targets contestants with some background in software engineering. Thus, this format is unsuitable for short contests that target beginners.

We further compare competitive programming contests using Probeable Problems with two other types of competitive events: the well-known Capture the Flag contests (which are used in informal educational settings, competitions, and more recently, in formal education contexts [46]) and Competitive Debugging [36]. In all these types of contests, participants seek to develop an understanding of a given system by carefully choosing certain inputs and examining the system’s behavior before writing (or generating) code. Probeable Problems seem to require a mindset more akin to CTF, since contestants are trying to find inputs that expose previously

Write a function `max_profit()` that takes one argument: a list of integers, where **positive values** in the list represent the price of a stock on some day. Your function should return **the best** day to buy and sell the stock in order to earn the **maximum profit**.

If the list is empty, return None.

Otherwise, return a tuple (buy, sell) representing the buy/sell days.

The maximum profit cannot be negative, and sell must be strictly greater than buy.

If no such (buy, sell) exists, no profit is possible, and your function should return (0, 0).

If the maximum profit can be earned for multiple (buy, sell) days, return the pair for which $\text{sell} - \text{buy}$ is as small as possible.

If there are multiple such (buy, sell) days, return the pair for which sell is as small as possible.

Ignore non-positive integers in the list.

Figure 7: Problem 4. The task specification draws attention to “positive values” in the list (highlighted in plain yellow) but deliberately neglects details on handling non-positive values (highlighted in chiaroscuro yellow). The *specific* value to be returned when the list is empty (shown in the red box with dashes) is deliberately omitted. The specification also deliberately omits the return type (shown in the blue box), the definition of the grey-highlighted term “maximum profit” (shown in the grey box), the return value when only non-negative profit can be earned (shown in the red box), and the tie-break mechanism for the cyan-highlighted term “the best” (a complex, nested mechanism shown in the cyan box). CodeCheck: [\[oracle\]](#), [\[verifier\]](#)

Write a function `reorder()` that takes two arguments: a list of **numbers** and another **number**. Your function should return a list in which the numbers that are **smaller** than the given number appear before the numbers that are **larger** than the given number.

The position of any value in the list equal to the given number should remain unchanged.

Numbers are either int or float values, and they may include special values like NaN.

If the given number is NaN, return the list as is.

The position of any NaN in the list should remain unchanged.

Figure 8: Problem 5. The task specification draws attention to values that are “smaller” and “larger” than the given number (highlighted in plain yellow) but deliberately neglects details on handling values equal to the given number (highlighted in chiaroscuro yellow). The specification also deliberately omits a definition of the grey-highlighted terms (shown in the grey box) and the return value when the arguments involve NaN (shown in the red boxes). CodeCheck: [\[oracle\]](#), [\[verifier\]](#)

unknown or unexpected behaviors, whereas debugging usually begins with at least one known failure-inducing input.

2.3 Understanding the Specification

Probeable Problems explicitly require contestants to reason about the (inadequacies of the) given specification or requirements. This skill may be under-developed, even amongst professionals [31]. For instance, Aniche et al. [2] note that developers reasoning about the

adequacy of their test cases tend to focus on their source code, their test code, and its code coverage while often failing to consider the documented specification. Such a biased focus while composing test suites or test plans may reflect practices developed during their formal education, where emphasis is often given to test adequacy criteria such as branch coverage or mutation analysis (e.g., [9], [28]). In a fascinating study of graduate students tasked with testing a single problem, Doorn et al. [14] observed that the tendency to think from an “implementation-first” mindset can appear even when *no*

implementation is provided, resulting in a failure to adequately cover the problem space.

To counteract this mindset, Shin and Kazerouni [43] encourage a “requirements-first” approach to testing. Probeable Problems align well with this goal by encouraging contestants to ask clarifying questions related to the requirements.

Probeable Problems that fail to specify the behavior on certain inputs (particularly the *chiaroscuro* sub-type that draw attention to a subset of the input space) may be challenging for beginners, who often consider test inputs only for the happy path [3, 6, 16]. Similarly, Probeable Problems that omit details for certain “corner case” inputs may challenge beginners, who often fail to consider such inputs while write test cases [1, 6].

To help students develop their ability to write effective tests, Cordova et al. [10] and Wrenn et al. [48] have developed feedback mechanisms that provide students with positive reinforcement when they explore conceptually interesting regions within the input space. In a similar spirit, we have attempted to select conceptually interesting details to omit from our specifications while developing Probeable Problems for this study.

Our problem specifications contain some natural language that students must comprehend. Wrenn et al. showed that an oracle, Exemplar, can improve students’ comprehension of the specification [48, 49]. Further, since instructors may *inadvertently* introduce ambiguities into code writing specifications by neglecting certain details, Exemplar may be helpful in identifying such missing details [33]. Our approach differs from this line of research in two ways. First, we consciously and carefully introduce ambiguities by omitting key details (see Section 1.2). Second, Exemplar asks a student to specify both an input and the output they expect on that input. Since this oracle’s purpose is to assist students in comprehending the specification, it indicates whether or not the output expected by the student matches the desired output for that input. In contrast, the purpose of the oracle in a Probeable Problem is to resolve the ambiguities that have been deliberately introduced by omitting details. An Exemplar-like oracle may force contestants to blindly guess the desired output, which could be frustrating.

3 STUDY CONTEXT

As a first evaluation of Probeable Problems, we conducted a 2-hour programming contest for undergraduate Computer Science students from multiple institutions, where each student was an active member of their institution’s ACM student chapter. This contest was part of a two-day event series organized by ACM India and in each event, students were free to participate individually or in self-formed teams. A total of 27 teams participated in our contest, with each team consisting of students from the same institution. Before the start of the contest, teams were informed that we would request consent to use their anonymized contest data for research purposes at the end of the contest.

3.1 Contest Format

In the first 20 minutes, we addressed students as a group to discuss the format and rules for our competitive programming contest. To begin with, we clarified that teams were free to use AI code-generation tools (e.g., ChatGPT, Codeium, or GitHub Copilot) on

some or all problems as they wished. Next, we introduced them to the idea of Probeable Problems. To avoid introducing unfamiliar terms such as ‘oracle’ and ‘Probeable Problems’, we instead referred to these as ‘the client’ and ‘Ask the Client Questions’ respectively.

As a demonstration, we opened the oracle link for the Practice Problem (Figure 3). To ensure that all contestants were clear about the usual meaning of the term ‘palindrome’ and to familiarize them with the oracle, we demonstrated the usage of its interface. Each oracle is initialized with an innocuous input that does not reveal any omitted details. For this demonstration, the initial input was the palindrome ‘racecar’. We demonstrated the oracle on two additional innocuous inputs: ‘abc’ (return value: False) and ‘abba’ (return value: True).

Next, we showed contestants how to create an initial solution for this problem using an AI code-generation model, Codeium. This model is easy to access via a web-interface⁷ without requiring any registration. Codeium unsurprisingly suggested Python code that returned the Boolean result of comparing the input string `s` directly against its reverse `s[::-1]`.

We then asked contestants to suggest inputs on which we could seek clarifications from the oracle. There were two immediate suggestions made by different students: the singleton letter ‘a’ and the empty string. Before querying the oracle on these inputs, we conducted a voice vote on what they expected the answers to be on these inputs based on (a) the specification, and (b) the code generated by Codeium. The overwhelming response was True in both cases, and we demonstrated that the oracle agrees with this verdict. After a few seconds, another contestant suggested ‘Racecar’ to “check for case sensitive inputs”. We showed that the oracle returns the somewhat unexpected result True on this input. We further noted that it was “reasonable for the client to want this behavior, but to forget to mention it” to the developer. We then asked them to “think like a developer and seek more clarity”. One student asked whether “spaces matter”, and we asked them to rephrase their question as a specific input for which we could ask the client for clarification. They suggested ‘race car’, and we showed that the oracle returned True in this case as well. Once again, we highlighted how this was a “reasonable but unstated” desire for the client to have.

We then asked a question of our own: “How do we know that we are done?” To answer this, we pointed out that contestants would be given a link to a *verifier* for each problem after submitting their first attempts. We demonstrated the verifier for the Practice Problem, whose link is given in the caption of Figure 3. (Similar links appear in the captions of the contest Problems.) We pointed out that each verifier can be used as often as needed, and it will run their code against a set of private test cases (on the basis of which the contest would be judged). However, we clarified that verifier merely indicated the *number* of passing test cases, but not the input(s) on which the code failed.

3.2 Contest Rules

In the last 10 minutes, we gave contestants specific instructions: they were to work in their current teams on 5 “Ask the Client” questions for 90 minutes, and we reiterated they were allowed to

⁷<https://codeium.com/playground>

	D	B	R	T
P1: least_positive_index		0.3		0.26
P2: first_positive_integer	0.12	0	0.59	
P3: min_freq				0.57
P4: max_profit	0.46	0.07	0.73	0
P5: reorder	0	0		

Figure 9: Heatmap of the performance of submissions for each problem against missing details. ‘D’, ‘B’, ‘R’, and ‘T’ respectively represent the four types of omissions: *Definitions*, *Behavior on certain inputs*, *Return type(s)*, and *Tie-break mechanism(s)*. Each cell (P, X) represents the fraction of all submissions for Problem P that correctly handled omissions of type X ; this cell is blank when Problem P does not have an omission of type X .

use any code-generation tools (including Codeium, ChatGPT, and GitHub Copilot). We provided them with a link to a Google Form for their first attempts, which contained links to the oracles for each problem. Thereafter, they would be pointed to a second Google Form containing links to both the oracles and the verifiers, and were free to submit second attempts for any of their failing solutions. After submitting the second form, they would be asked whether *all* members granted consent for us to use their anonymized data for research purposes, or whether *at least one* member did not grant consent. We clarified that their answer would have no bearing on the contest. Except for one team with 2 students, all teams granted consent. Thus, our study is based on 67 students in 26 teams (average: 2.6 students per team).

4 RESULTS

There were 17 teams with 2 students, 5 teams with 3 students, 2 teams with 4 students, and 1 team each with 1, 5, and 6 students.

4.1 RQ1: Success at Determining Missing Details

Overall, we observed that the contestants could determine only about 20% of the missing details. On average, submissions could only handle 3.88 missing details out of 19 across the five problems. Instead of creating test cases, we evaluated the submitted code using Hypothesis⁸, a popular Python library for property-based testing. We created targeted equivalence tests to evaluate each submission against each omission (one at a time). For each test, we carefully specified the class of inputs to test the submission against, ensuring that each test was independent. To ensure that our tests explore the full space of inputs, we ran a “catch-all” unrestricted test on submissions after running our targeted tests. However, we

found that our targeted tests were collectively exhaustive since no submission passed on all targeted tests but failed on the unrestricted test.⁹

For Problem 1, around 30% of the submissions correctly handled the behavior on inputs with more than one instance of the smallest positive integer. On the other hand, while only 26% of the submissions correctly handled the behavior on inputs with no positive integers, 63% of the submissions could at least handle a simple case (with the input `[]`). This was possibly because the output in the simplest case (which is `-1`) is conventionally the output on the entire class of such inputs and thus not only could contestants incorrectly assume the intended behavior, but also AI tools are very likely to suggest such code. There was one correct submission to this problem.

For Problem 2, in order to determine whether students figured out the omitted definition of the ‘first positive integer’, we checked the contestant code in three steps. We first checked how they handled single words that contained at most a single positive digit (as its suffix). We then checked how they handled single words that contained several valid prefixes. Finally, we checked how they handled strings containing multiple words. With this characterization, we split the original omission into three sub-omissions – one corresponding to each of these checks. None of the submissions correctly handled the first, around 66% correctly handled the second, and 45% correctly handled the third. Around 59% of the submissions returned the correct type of values, i.e., integers. Although 59% of the submissions correctly handled the simplest input (the empty string) without any positive integer, none of the submissions could generalize the notion to the entire class of such inputs.

For Problem 3, 57% of the submissions correctly handled inputs with multiple least frequent integers – both in the simplest case (e.g., `[1, 0]`) as well as in the general case. It was interesting to note that 93% of the submissions correctly handled a similar simple input `[0, 1]` since many contestants either incorrectly assumed that the intended behavior was to pick the first such number in the list, or used an AI tool that produced code with this conventional assumption.

For Problem 4 as well, we split the omission of definition into two separate omissions for the purpose of evaluating contestant code – the first being that the profit must be non-negative, and the second being the condition `sell > buy`. We found that while around 87% of the submissions correctly determined the first property, all the submissions correctly determined the second. Only 37%, 10%, and 33% of the submissions respectively could handle inputs with no non-negative profit, at most zero profit, and with negative list items. Only 30% of the submissions correctly returned None for the empty list.

For Problem 5, while none of the contestants correctly handled any of the omissions, we found that 12% of the submissions could handle a simple case with NaN key (with the list `[0, 1]`), 4% could handle a simple case with NaN in the list (with the list `[2, nan, 0]` and the key 1), and 28% could handle a simple case of key values appearing in the list (with the list `[2, 1, 0]` and the key 1).

⁸<https://hypothesis.readthedocs.io>

⁹Our evaluation scripts, together with the anonymized contestant submissions as well as the analysis, are available at: <https://github.com/mrigankpawagi/ProbeableProblems>

4.2 RQ2: Leveraging Verifiers

We received a total of 12 second submissions: 2 submissions for Problem 1, 4 for Problem 2, none for Problem 3, 5 for Problem 4, and 1 for Problem 5. In problem 1, one team determined the correct way to handle lists with no positive integers in their second submission. This team correctly handled the simple case in their first submission, and thus feedback from the verifier helped them successfully apply inductive reasoning. In problem 4, one team was able to leverage the verifier to correctly handle inputs with zero profit. On closer observation by taking the diff between their two submissions, we found that they corrected their code by identifying an incorrect $<$ in their code which should have been $\leq$, thereby resolving a tie-break correctly. Three other second submissions for Problem 4 were functionally equivalent to the respective first submissions. All other second submissions handled at least one simple case or class of inputs incorrectly which the respective first submission had handled correctly, and did not correctly handle any new input or input class.

5 LIMITATIONS

Before discussing our findings, we acknowledge two important limitations of our work. First, our categorization of the details omitted during the design of Probeable Problems (Section 1.2) is *ad hoc*. While these categories seem intuitive to us, we believe that a firmer basis grounded in cognitive and/or linguistic principles should be used (or developed).

Second, while our analysis has focused on the *code* generated by contestants, their interactions with the oracles and verifiers have not been captured. For this study, we hosted these elements on CodeCheck because the constraints of the host event (of which our contest was a part) left us inadequate time to create logins for each team on a server where we could collect data. (The CodeCheck server does not log usage data by design.) We are planning a future study where we will gather and analyze this rich interaction data.

6 DISCUSSION

The students who participated in our contest were active members of their institution’s ACM student chapter, and all teams worked on the problems for at least 85 out of the 90 minutes allotted. Thus, it is reasonable to assume that a good-faith effort was made by most teams. Given this, the low overall performance of teams (Figure 9) suggests that the contest problems were too challenging for the time allotted. Going forward, we believe that changes to the design of Probeable Problems and to the contest format are necessary. Further, we believe there are interesting avenues to explore connections between Probeable Problems in formal educational contexts as well.

6.1 Reflections on the Contest and Probeable Problem Design

Three changes to the design of the contest are immediately warranted. First, providing contestants with a single Practice Problem with just one type of omission appears inadequate. In future iterations of this contest, we will develop a richer set of Practice Problems both for illustration prior to the event and, following [36], to help contestants prepare better for the event. Second, given the

poor performance on Problem 5 and the very few submissions overall for the second attempt (where the performance on 7 out of 12 resubmissions was worse), it is clear that contestants faced a severe time-crunch. We will organize future contests with fewer problems or provide contestants with more time. Third, we are exploring ways in which the verifiers can provide a limited amount of detailed feedback (e.g., a small number of counter-examples) when the initial submission fails. We believe that this, too, will boost the number of resubmissions.

With regards to the design of Probeable Problems, we believe that our decision to eliminate *all* details from the complex definition in Problem 2 and from the complex tie-breaker in Problem 4 was poor. Further, our oracle at present only provides answers to specific inputs. Once a contestant has formulated an inductive guess for the expected behavior on a general class of inputs, we would like the oracle to be able to handle this general query. Two options that seem promising are: (1) using the syntax of property-based testing [8], and (2) using natural language.

6.2 Developing the ability to ask clarifying questions

Real-world programming task specifications may be incomplete or may contain statements that are *ambiguous* i.e., they can be interpreted in multiple ways [5, 26]. Developers who fail to recognize these deficiencies in the specification may write code that is defective from the client’s perspective. Rectifying such defects has the potential to cause expensive delays and may even lead to further defects [18, 21]. However, a recent study suggests that no special effort is needed to detect such ambiguities “beyond the normal stakeholders thoroughly discussing the [specification]” [37]. We believe that CS students must develop the ability to thoroughly discuss specifications, particularly by identifying potential ambiguities and asking clarifying questions. We view Probeable Problems as one way to develop this ability.

On the other hand, we are concerned that AI tools may be able to surpass humans in this ability. While the performance of LLMs for code-generation have improved significantly, state-of-the-art models are not yet able to match skillful developers [44] or participants in competitive programming contests [25]. To improve the performance of these models, a promising line of research is to generate code in careful phases. For instance, ChatDev [35] mimics the classical 4-stage waterfall model. When presented with a specification that is vague and high-level, ChatDev adopts an LLM-based virtual role and asks clarifying questions to generate a more detailed specification. Similar ideas are used by MetaGPT [22] and Alphacodeium [38]. A simpler approach by Pawagi and Kumar can solve some Probeable Problems by leveraging existing AI code-generation tools such as GitHub Copilot [32]. Specifically, their tool repeatedly invokes the underlying LLM and exploits the fact that missing details are often filled in differently in different solutions. Thus, by identifying inputs on which these solutions differ, it automatically suggests inputs for which clarifications can be sought from a Probeable Problem oracle.

7 CONCLUSIONS AND FUTURE WORK

This paper has introduced a novel type of code-writing problem called a Probeable Problem. The primary motivation for our work is to develop a viable substitute for beginner-level competitive programming problems when the use of AI code-generation tools is permitted. We have showed that carefully constructed Probeable Problems cannot be solved by modern, easy-to-access AI code-generation tools. Further, we believe that Probeable Problems can help students develop an important ability: to ask clarifying questions when presented with incomplete or ambiguous task specifications.

As noted in Section 2.3, we see interesting connections between the skills needed to solve Probeable Problems and prior research in helping students write good tests. We believe that this is a fruitful area of research. Further, we believe that there are interesting questions to explore on how well students comprehend the code [24] in a Probeable Problem when they are writing it manually versus using an AI code-generation tool. For example, if the contestants in each vignette in Section 1.1 had written their code manually, would they be more likely to switch between code-level and task-level thinking [7]? Further, the oracles we have considered in this study are “unconstrained” i.e., they immediately reveal the expected output for *any* input provided by the contestant. Could such oracles adversely impact the ability of contestants to develop their understanding of even the non-ambiguous aspects of the given task specification? If so, the oracles can be constrained to provide immediate answers only when the input is associated with an omitted detail. Finally, we believe that Probeable Problems may be interesting for more complex code writing tasks (beyond CS1).

ACKNOWLEDGMENTS

We thank the Kotak-IISc AI-ML Centre at IISc Bangalore for supporting this research.

REFERENCES

- [1] Mauricio Aniche, Felienne Hermans, and Arie van Deursen. 2019. Pragmatic Software Testing Education. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (Minneapolis, MN, USA) (SIGCSE '19). Association for Computing Machinery, New York, NY, USA, 414–420. <https://doi.org/10.1145/3287324.3287461>
- [2] Mauricio Aniche, Christoph Treude, and Andy Zaidman. 2022. How Developers Engineer Test Cases: An Observational Study. *IEEE Transactions on Software Engineering* 48, 12 (2022), 4925–4946. <https://doi.org/10.1109/TSE.2021.3129889>
- [3] Gina R. Bai, Justin Smith, and Kathryn T. Stolee. 2021. How Students Unit Test: Perceptions, Practices, and Pitfalls. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1* (Virtual Event, Germany) (ITiCSE '21). Association for Computing Machinery, New York, NY, USA, 248–254. <https://doi.org/10.1145/3430665.3456368>
- [4] Brett A. Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. 2023. Programming Is Hard - Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (<conf-loc>, <city>Toronto ON</city>, <country>Canada</country>, </conf-loc>) (SIGCSE 2023). Association for Computing Machinery, New York, NY, USA, 500–506. <https://doi.org/10.1145/3545945.3569759>
- [5] Daniel M. Berry and Erik Kamsties. 2004. *Ambiguity in Requirements Specification*. Springer US, Boston, MA, 7–44. https://doi.org/10.1007/978-1-4615-0465-8_2
- [6] Lex Bijlsma, Niels Doorn, Harrie Passier, Harold Pootjes, and Sylvia Stuurman. 2021. How do students test software units?. In *Proceedings of the 43rd International Conference on Software Engineering: Joint Track on Software Engineering Education and Training* (Virtual Event, Spain) (ICSE-JSEET '21). IEEE Press, 189–198. <https://doi.org/10.1109/ICSE-SEET52601.2021.00029>
- [7] Francisco Enrique Vicente Castro and Kathi Fisler. 2020. Qualitative Analyses of Movements Between Task-level and Code-level Thinking of Novice Programmers. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (Portland, OR, USA) (SIGCSE '20). Association for Computing Machinery, New York, NY, USA, 487–493. <https://doi.org/10.1145/3328778.3366847>
- [8] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. *SIGPLAN Not.* 35, 9 (sep 2000), 268–279. <https://doi.org/10.1145/357766.351266>
- [9] Peter J. Clarke, Debra Davis, Tariq M. King, Jairo Pava, and Edward L. Jones. 2014. Integrating Testing into Software Engineering Courses Supported by a Collaborative Learning Environment. *ACM Trans. Comput. Educ.* 14, 3, Article 18 (oct 2014), 33 pages. <https://doi.org/10.1145/2648787>
- [10] Lucas Cordova, Jeffrey Carver, Noah Gershmel, and Gursimran Walia. 2021. A Comparison of Inquiry-Based Conceptual Feedback vs. Traditional Detailed Feedback Mechanisms in Software Testing Education: An Empirical Investigation. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education* (Virtual Event, USA) (SIGCSE '21). Association for Computing Machinery, New York, NY, USA, 87–93. <https://doi.org/10.1145/3408877.3432417>
- [11] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) (SIGCSE 2023). Association for Computing Machinery, New York, NY, USA, 1136–1142. <https://doi.org/10.1145/3545945.3569823>
- [12] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. 2024. Prompt Problems: A New Programming Exercise for the Generative AI Era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (<conf-loc>, <city>Portland</city>, <state>OR</state>, <country>USA</country>, </conf-loc>) (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA, 296–302. <https://doi.org/10.1145/3626252.3630909>
- [13] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2023. Computing Education in the Era of Generative AI. arXiv:2306.02608 [cs.CY]
- [14] Niels Doorn, Tanja E. J. Vos, Beatriz Marín, Harrie Passier, Lex Bijlsma, and Silvio Cacace. 2021. Exploring students' sensemaking of test case design. An initial study. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. 1069–1078. <https://doi.org/10.1109/QRS-C55045.2021.00161>
- [15] Lyn Dupré. 1995. *Bugs in writing, a guide to debugging your prose*. ACM Press/Addison-Wesley Publishing Co.
- [16] Stephen H. Edwards and Zalia Shams. 2014. Do student programmers all tend to write the same software tests?. In *Proceedings of the 2014 Conference on Innovation & Technology in Computer Science Education* (Uppsala, Sweden) (ITiCSE '14). Association for Computing Machinery, New York, NY, USA, 171–176. <https://doi.org/10.1145/2591708.2591757>
- [17] Saad Ezzi, Sallam Abualhaija, Chetan Arora, Mehrdad Sabetzadeh, and Lionel C Briand. 2021. Using domain-specific corpora for improved handling of ambiguity in requirements. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1485–1497.
- [18] Alessio Ferrari and Andrea Esuli. 2019. An NLP approach for cross-domain ambiguity detection in requirements engineering. *Automated Software Engineering* 26, 3 (2019), 559–598.
- [19] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference* (<conf-loc>, <city>Virtual Event</city>, <country>Australia</country>, </conf-loc>) (ACE '22). Association for Computing Machinery, New York, NY, USA, 10–19. <https://doi.org/10.1145/3511861.3511863>
- [20] James Finnie-Ansley, Paul Denny, Andrew Luxton-Reilly, Eddie Antonio Santos, James Prather, and Brett A. Becker. 2023. My AI Wants to Know if This Will Be on the Exam: Testing OpenAI's Codex on CS2 Programming Exercises. In *Proceedings of the 25th Australasian Computing Education Conference* (<conf-loc>, <city>Melbourne</city>, <state>VIC</state>, <country>Australia</country>, </conf-loc>) (ACE '23). Association for Computing Machinery, New York, NY, USA, 97–104. <https://doi.org/10.1145/3576123.3576134>
- [21] Donald C Gause and Gerald M Weinberg. 1989. Exploring requirements. *Dorset House* (1989), 249.
- [22] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352 [cs.AI]
- [23] Yiming Huang, Zhenghao Lin, Xiao Liu, Yeyun Gong, Shuai Lu, Fangyu Lei, Yaobo Liang, Yelong Shen, Chen Lin, Nan Duan, and Weizhu Chen. 2023. Competition-Level Problems are Effective LLM Evaluators. arXiv:2312.02143 [cs.CL]
- [24] Cruz Izu, Carsten Schulte, Ashish Aggarwal, Quintin Cutts, Rodrigo Duran, Mirela Gutica, Birte Heinemann, Eileen Kraemer, Violetta Lonati, Claudio Mirolo, and Renske Weeda. 2019. Fostering Program Comprehension in Novice Programmers - Learning Activities and Learning Trajectories. In *Proceedings of the Working Group*

- Reports on Innovation and Technology in Computer Science Education* (Aberdeen, Scotland UK) (*ITICSE-WGR '19*). Association for Computing Machinery, New York, NY, USA, 27–52. <https://doi.org/10.1145/3344429.3372501>
- [25] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Live-CodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv:2403.07974* [cs.SE]
- [26] Erik Kamsties. 2005. *Understanding Ambiguity in Requirements Engineering*. Springer Berlin Heidelberg, Berlin, Heidelberg, 245–266. https://doi.org/10.1007/3-540-28244-0_11
- [27] Erik Kamsties and Barbara Peach. 2000. Taming ambiguity in natural language requirements. In *Proceedings of the Thirteenth international conference on Software and Systems Engineering and Applications*, Vol. 1315.
- [28] Ayaan M. Kazerouni, Clifford A. Shaffer, Stephen H. Edwards, and Francisco Servant. 2019. Assessing Incremental Testing Practices and Their Impact on Project Outcomes. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (Minneapolis, MN, USA) (*SIGCSE '19*). Association for Computing Machinery, New York, NY, USA, 407–413. <https://doi.org/10.1145/3287324.3287366>
- [29] Benjamin L Kovitz. 1998. *Practical software requirements: a manual of content and style*. Manning Publications Co.
- [30] Dino Mandrioli, Stephen Fickas, Carlo A. Furia, Mehdi Jazayeri, Matteo Rossi, and Michal Young. 2010. SCORE: the first student contest on software engineering. *SIGSOFT Softw. Eng. Notes* 35, 4 (jul 2010), 24–30. <https://doi.org/10.1145/1811226.1811240>
- [31] Ray Offen. 2002. Domain understanding is the key to successful system development. *Requirements engineering* 7, 3 (2002), 172.
- [32] Mrigank Pawagi and Viraj Kumar. 2023. GuardRails: Automated Suggestions for Clarifying Ambiguous Purpose Statements. In *Proceedings of the 16th Annual ACM India Compute Conference* (<conf-loc>, <city>Hyderabad</city>, <country>India</country>, </conf-loc>) (*COMPUTE '23*). Association for Computing Machinery, New York, NY, USA, 55–60. <https://doi.org/10.1145/3627217.3627234>
- [33] Siddhartha Prasad, Ben Greenman, Tim Nelson, John Wrenn, and Shriram Krishnamurthi. 2022. Making Hay from Wreaths: A Classsourcing Method to Identify Misconceptions. In *Proceedings of the 22nd Koli Calling International Conference on Computing Education Research* (Koli, Finland) (*Koli Calling '22*). Association for Computing Machinery, New York, NY, USA, Article 2, 7 pages. <https://doi.org/10.1145/3564721.3564726>
- [34] James Prather, Paul Denny, Juho Leinonen, David H. Smith IV au2, Brent N. Reeves, Stephen MacNeil, Brett A. Becker, Andrew Luxton-Reilly, Thezyrie Amarouche, and Bailey Kimmel. 2024. Interactions with Prompt Problems: A New Way to Teach Programming with Large Language Models. *arXiv:2401.10759* [cs.HC]
- [35] Chen Qian, Xin Cong, Wei Liu, Cheng Yang, Weize Chen, Yusheng Su, Yufan Dang, Jiahao Li, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. Communicative Agents for Software Development. *arXiv:2307.07924* [cs.SE]
- [36] Patrick Rein, Tom Beckmann, Leonard Geier, Toni Mattis, and Robert Hirschfeld. 2022. Competitive Debugging: Toward Contests Promoting Debugging as a Skill. In *Proceedings of the 2022 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Auckland, New Zealand) (*Onward! 2022*). Association for Computing Machinery, New York, NY, USA, 172–184. <https://doi.org/10.1145/3563835.3567665>
- [37] Cristina Ribeiro and Daniel Berry. 2020. The prevalence and severity of persistent ambiguity in software requirements specifications: Is a special effort needed to find them? *Science of Computer Programming* 195 (2020), 102472. <https://doi.org/10.1016/j.scico.2020.102472>
- [38] Tal Ridnik, Dedy Kreda, and Itamar Friedman. 2024. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *arXiv:2401.08500* [cs.LG]
- [39] Vidhu Rojit, Sindhu R. Pai, Shruthi Kaivalya, and Viraj Kumar. 2016. Visual Specifications for Web-Application Programming Assignments. In *2016 IEEE Eighth International Conference on Technology for Education (T4E)*. 50–53. <https://doi.org/10.1109/T4E.2016.019>
- [40] Jaromir Savelka, Arav Agarwal, Marshall An, Chris Bogart, and Majd Sakr. 2023. Thrilled by Your Progress! Large Language Models (GPT-4) No Longer Struggle to Pass Assessments in Higher Education Programming Courses. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1* (<conf-loc>, <city>Chicago</city>, <state>IL</state>, <country>USA</country>, </conf-loc>) (*ICER '23*). Association for Computing Machinery, New York, NY, USA, 78–92. <https://doi.org/10.1145/3568813.3600142>
- [41] G. Michael Schneider. 1978. The Introductory Programming Course in Computer Science: Ten Principles. In *Papers of the SIGCSE/CSA Technical Symposium on Computer Science Education* (Detroit, Michigan) (*SIGCSE '78*). Association for Computing Machinery, New York, NY, USA, 107–114. <https://doi.org/10.1145/990555.990598>
- [42] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. *arXiv:2310.11324* [cs.CL]
- [43] Austin M. Shin and Ayaan M. Kazerouni. 2024. A Model of How Students Engineer Test Cases With Feedback. *ACM Trans. Comput. Educ.* 24, 1, Article 1 (jan 2024), 31 pages. <https://doi.org/10.1145/3628604>
- [44] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Neumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [45] Varshini Venkatesh, Vaishnavi Venkatesh, and Viraj Kumar. 2023. Evaluating Copilot on CS1 Code Writing Problems with Suppressed Specifications. In *Proceedings of the 16th Annual ACM India Compute Conference* (<conf-loc>, <city>Hyderabad</city>, <country>India</country>, </conf-loc>) (*COMPUTE '23*). Association for Computing Machinery, New York, NY, USA, 104–107. <https://doi.org/10.1145/3627217.3627235>
- [46] Jan Vykopal, Valdemar Švábenský, and Ee-Chien Chang. 2020. Benefits and Pitfalls of Using Capture the Flag Games in University Courses. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (Portland, OR, USA) (*SIGCSE '20*). Association for Computing Machinery, New York, NY, USA, 752–758. <https://doi.org/10.1145/3328778.3366893>
- [47] Michel Wermelinger. 2023. Using GitHub Copilot to Solve Simple Programming Problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) (*SIGCSE 2023*). Association for Computing Machinery, New York, NY, USA, 172–178. <https://doi.org/10.1145/3545945.3569830>
- [48] John Wrenn and Shriram Krishnamurthi. 2019. Executable Examples for Programming Problem Comprehension. In *Proceedings of the 2019 ACM Conference on International Computing Education Research* (Toronto ON, Canada) (*ICER '19*). Association for Computing Machinery, New York, NY, USA, 131–139. <https://doi.org/10.1145/3291279.3339416>
- [49] John Wrenn and Shriram Krishnamurthi. 2021. Reading Between the Lines: Student Help-Seeking for (Un) Specified Behaviors. In *Proceedings of the 21st Koli Calling International Conference on Computing Education Research*. 1–6.
- [50] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyan Wu, Yihao Zhao, Chen Yang, Shihe Wang, Qiyang Zhang, Zhenyan Lu, Li Zhang, Shanguang Wang, Yuanchun Li, Yunxin Liu, Xin Jin, and Xuanzhe Liu. 2024. A Survey of Resource-efficient LLM and Multimodal Foundation Models. *arXiv:2401.08092* [cs.LG]